

```
"""
```

```
    |
    |                                     |
    |         PocketMoneyHub Bot         |
    |                                     |
    |         Telegram SMS Management System         |
    |                                     |
    |         Developed by PocketMoneyHub         |
    |                                     |
    |                                     |
    |         GitHub: https://github.com/hsh34811-hash         |
    |                                     |
    |         Bot: @PocketMoneyNumber_bot         |
    |                                     |
    |         Channel: https://t.me/+OQ0CTFKdFFtmZDk1         |
    |                                     |
    |                                     |
    |         Copyright © 2026 PocketMoneyHub - All Rights Reserved         |
    |                                     |
    |
```

```
"""
```

```
import time
import requests
import json
import re
import os
from datetime import datetime, date, timedelta
from urllib.parse import quote_plus
from pathlib import Path
import sqlite3
import telebot
from telebot import types
import threading
import traceback
import random
import itertools
import logging
import asyncio
import httpx
from bs4 import BeautifulSoup
from urllib.parse import urljoin

# PocketMoneyHub | إعداد اللوحة الوحيدة (iVasms)

IVASMS_DASHBOARD = {
    "name": "iVasms",
    "type": "ivasms",
    "login_url": "https://ivas.tempnum.qzz.io/login",
    "base_url": "https://ivas.tempnum.qzz.io",
    "sms_api_endpoint": "https://ivas.tempnum.qzz.io/portal/sms/received/getsms",
    "username": "YOUR_EMAIL_HERE",
```

```

    "password": "YOUR_PASSWORD_HERE",
    "session": requests.Session(),
    "is_logged_in": False,
    "cookies": None,
    "csrf_token": None,
    "last_check": None
}

# PocketMoneyHub | إعدادات عامة
USERNAME = "YOUR_USERNAME_HERE"
PASSWORD = "YOUR_SITE_PASSWORD_HERE"
BOT_TOKEN = os.environ.get("BOT_TOKEN", "")
CHAT_IDS = [
    "YOUR_CHAT_ID_HERE",
]

REFRESH_INTERVAL = 2 # ⚡ تقليل من 3 إلى 2 ثانية لسحب أسرع
TIMEOUT = 100
MAX_RETRIES = 5
RETRY_DELAY = 5

# مؤشرات الأعمدة للوحة التلقيدية (كما في الأصل)
IDX_DATE = 0
IDX_NUMBER = 2
IDX_SMS = 5
SENT_MESSAGES_FILE = "sent_messages.json"

ADMIN_IDS = [0000000000]
DB_PATH = "bot.db"
FORCE_SUB_CHANNEL = None
FORCE_SUB_ENABLED = False
BOT_ACTIVE = True

if not BOT_TOKEN:
    raise SystemExit("❌ BOT_TOKEN must be set")
if not CHAT_IDS:
    raise SystemExit("❌ CHAT_IDS must be configured")
if not USERNAME or USERNAME == "YOUR_USERNAME_HERE" or not PASSWORD or PASSWORD == "YOUR_SITE_PASSWORD_HERE":
    print("⚠️ WARNING: SITE_USERNAME and SITE_PASSWORD not set")
    print("⚠️ Bot will continue but login may fail")

# PocketMoneyHub | رموز الدول
COUNTRY_CODES = {
    "1": ("USA/Canada", "🇺🇸", "US"),
    "7": ("Russia", "🇷🇺", "RU"),
    "20": ("Egypt", "🇪🇬", "EG"),
    "27": ("South Africa", "🇿🇦", "ZA"),
    "30": ("Greece", "🇬🇷", "GR"),

```

"31": ("Netherlands", "", "NL"),
"32": ("Belgium", "", "BE"),
"33": ("France", "", "FR"),
"34": ("Spain", "", "ES"),
"36": ("Hungary", "", "HU"),
"39": ("Italy", "", "IT"),
"40": ("Romania", "", "RO"),
"41": ("Switzerland", "", "CH"),
"42": ("????", " ? ", "??"),
"43": ("Austria", "", "AT"),
"44": ("United Kingdom", "", "UK"),
"45": ("Denmark", "", "DK"),
"46": ("Sweden", "", "SE"),
"47": ("Norway", "", "NO"),
"48": ("Poland", "", "PL"),
"49": ("Germany", "", "DE"),

"51": ("Peru", "", "PE"),
"52": ("Mexico", "", "MX"),
"53": ("Cuba", "", "CU"),
"54": ("Argentina", "", "AR"),
"55": ("Brazil", "", "BR"),
"56": ("Chile", "", "CL"),
"57": ("Colombia", "", "CO"),
"58": ("Venezuela", "", "VE"),

"60": ("Malaysia", "", "MY"),
"61": ("Australia", "", "AU"),
"62": ("Indonesia", "", "ID"),
"63": ("Philippines", "", "PH"),
"64": ("New Zealand", "", "NZ"),
"65": ("Singapore", "", "SG"),
"66": ("Thailand", "", "TH"),

"81": ("Japan", "", "JP"),
"82": ("South Korea", "", "KR"),
"84": ("Vietnam", "", "VN"),
"86": ("China", "", "CN"),

"90": ("Turkey", "", "TR"),
"91": ("India", "", "IN"),
"92": ("Pakistan", "", "PK"),
"93": ("Afghanistan", "", "AF"),
"94": ("Sri Lanka", "", "LK"),
"95": ("Myanmar", "", "MM"),
"98": ("Iran", "", "IR"),

"211": ("South Sudan", "", "SS"),
"212": ("Morocco", "", "MA"),
"213": ("Algeria", "", "DZ"),
"216": ("Tunisia", "", "TN"),
"218": ("Libya", "", "LY"),

"220": ("Gambia", "", "GM"),
"221": ("Senegal", "", "SN"),
"222": ("Mauritania", "", "MR"),
"223": ("Mali", "", "ML"),
"224": ("Guinea", "", "GN"),
"225": ("Ivory Coast", "", "CI"),
"226": ("Burkina Faso", "", "BF"),
"227": ("Niger", "", "NE"),
"228": ("Togo", "", "TG"),
"229": ("Benin", "", "BJ"),

"230": ("Mauritius", "", "MU"),
"231": ("Liberia", "", "LR"),
"232": ("Sierra Leone", "", "SL"),
"233": ("Ghana", "", "GH"),
"234": ("Nigeria", "", "NG"),
"235": ("Chad", "", "TD"),
"236": ("Central African Rep", "", "CF"),
"237": ("Cameroon", "", "CM"),
"238": ("Cape Verde", "", "CV"),
"239": ("Sao Tome", "", "ST"),
"240": ("Equatorial Guinea", "", "GQ"),
"241": ("Gabon", "", "GA"),
"242": ("Congo", "", "CG"),
"243": ("DR Congo", "", "CD"),
"244": ("Angola", "", "AO"),
"245": ("Guinea-Bissau", "", "GW"),

"248": ("Seychelles", "", "SC"),
"249": ("Sudan", "", "SD"),
"250": ("Rwanda", "", "RW"),
"251": ("Ethiopia", "", "ET"),
"252": ("Somalia", "", "SO"),
"253": ("Djibouti", "", "DJ"),
"254": ("Kenya", "", "KE"),
"255": ("Tanzania", "", "TZ"),
"256": ("Uganda", "", "UG"),
"257": ("Burundi", "", "BI"),
"258": ("Mozambique", "", "MZ"),
"260": ("Zambia", "", "ZM"),
"261": ("Madagascar", "", "MG"),

"262": ("Reunion", "", "RE"),
"263": ("Zimbabwe", "", "ZW"),
"264": ("Namibia", "", "NA"),
"265": ("Malawi", "", "MW"),
"266": ("Lesotho", "", "LS"),
"267": ("Botswana", "", "BW"),
"268": ("Eswatini", "", "SZ"),
"269": ("Comoros", "", "KM"),

"350": ("Gibraltar", "", "GI"),
"351": ("Portugal", "", "PT"),
"352": ("Luxembourg", "", "LU"),
"353": ("Ireland", "", "IE"),
"354": ("Iceland", "", "IS"),
"355": ("Albania", "", "AL"),
"356": ("Malta", "", "MT"),
"357": ("Cyprus", "", "CY"),
"358": ("Finland", "", "FI"),
"359": ("Bulgaria", "", "BG"),
"370": ("Lithuania", "", "LT"),
"371": ("Latvia", "", "LV"),
"372": ("Estonia", "", "EE"),
"373": ("Moldova", "", "MD"),
"374": ("Armenia", "", "AM"),
"375": ("Belarus", "", "BY"),
"376": ("Andorra", "", "AD"),
"377": ("Monaco", "", "MC"),
"378": ("San Marino", "", "SM"),
"380": ("Ukraine", "", "UA"),
"381": ("Serbia", "", "RS"),
"382": ("Montenegro", "", "ME"),
"383": ("Kosovo", "", "XK"),
"385": ("Croatia", "", "HR"),
"386": ("Slovenia", "", "SI"),
"387": ("Bosnia", "", "BA"),
"389": ("North Macedonia", "", "MK"),

"420": ("Czech Republic", "", "CZ"),
"421": ("Slovakia", "", "SK"),
"423": ("Liechtenstein", "", "LI"),

"500": ("Falkland Islands", "", "FK"),
"501": ("Belize", "", "BZ"),
"502": ("Guatemala", "", "GT"),
"503": ("El Salvador", "", "SV"),
"504": ("Honduras", "", "HN"),
"505": ("Nicaragua", "", "NI"),

"506": ("Costa Rica", "", "CR"),
"507": ("Panama", "", "PA"),
"509": ("Haiti", "", "HT"),

"591": ("Bolivia", "", "BO"),
"592": ("Guyana", "", "GY"),
"593": ("Ecuador", "", "EC"),
"595": ("Paraguay", "", "PY"),
"597": ("Suriname", "", "SR"),
"598": ("Uruguay", "", "UY"),

"670": ("Timor-Leste", "", "TL"),
"673": ("Brunei", "", "BN"),
"674": ("Nauru", "", "NR"),
"675": ("Papua New Guinea", "", "PG"),
"676": ("Tonga", "", "TO"),
"677": ("Solomon Islands", "", "SB"),
"678": ("Vanuatu", "", "VU"),
"679": ("Fiji", "", "FJ"),
"680": ("Palau", "", "PW"),
"685": ("Samoa", "", "WS"),
"686": ("Kiribati", "", "KI"),
"687": ("New Caledonia", "", "NC"),
"688": ("Tuvalu", "", "TV"),
"689": ("French Polynesia", "", "PF"),
"691": ("Micronesia", "", "FM"),
"692": ("Marshall Islands", "", "MH"),

"850": ("North Korea", "", "KP"),
"852": ("Hong Kong", "", "HK"),
"853": ("Macau", "", "MO"),
"855": ("Cambodia", "", "KH"),
"856": ("Laos", "", "LA"),

"960": ("Maldives", "", "MV"),
"961": ("Lebanon", "", "LB"),
"962": ("Jordan", "", "JO"),
"963": ("Syria", "", "SY"),
"964": ("Iraq", "", "IQ"),
"965": ("Kuwait", "", "KW"),
"966": ("Saudi Arabia", "", "SA"),
"967": ("Yemen", "", "YE"),
"968": ("Oman", "", "OM"),
"970": ("Palestine", "", "PS"),
"971": ("UAE", "", "AE"),
"972": ("Israel", "", "IL"),
"973": ("Bahrain", "", "BH"),

```

"974": ("Qatar", "", "QA"),
"975": ("Bhutan", "", "BT"),
"976": ("Mongolia", "", "MN"),
"977": ("Nepal", "", "NP"),

"992": ("Tajikistan", "", "TJ"),
"993": ("Turkmenistan", "", "TM"),
"994": ("Azerbaijan", "", "AZ"),
"995": ("Georgia", "", "GE"),
"996": ("Kyrgyzstan", "", "KG"),
"998": ("Uzbekistan", "", "UZ"),
}

```

PocketMoneyHub | إدارة قاعدة البيانات

```

def get_setting(key):
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute("SELECT value FROM bot_settings WHERE key=?", (key,))
    row = c.fetchone()
    conn.close()
    return row[0] if row else None

def set_setting(key, value):
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute("REPLACE INTO bot_settings (key, value) VALUES (?, ?)", (key, value))
    conn.commit()
    conn.close()

```

PocketMoneyHub | إنشاء قاعدة البيانات

```

def init_db():
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute('''
        CREATE TABLE IF NOT EXISTS users (
            user_id INTEGER PRIMARY KEY,
            username TEXT,
            first_name TEXT,
            last_name TEXT,
            country_code TEXT,
            assigned_number TEXT,
            is_banned INTEGER DEFAULT 0,
            private_combo_country TEXT DEFAULT NULL
        )
    ''')
    c.execute('''
        CREATE TABLE IF NOT EXISTS combos (

```

```

        id INTEGER PRIMARY KEY AUTOINCREMENT,
        country_code TEXT,
        combo_index INTEGER DEFAULT 1,
        numbers TEXT,
        UNIQUE(country_code, combo_index)
    )
'''
c.execute('''
    CREATE TABLE IF NOT EXISTS otp_logs (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        number TEXT,
        otp TEXT,
        full_message TEXT,
        timestamp TEXT,
        assigned_to INTEGER
    )
'''
c.execute('''
    CREATE TABLE IF NOT EXISTS dashboards (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        base_url TEXT,
        ajax_path TEXT,
        login_page TEXT,
        login_post TEXT,
        username TEXT,
        password TEXT
    )
'''
c.execute('''
    CREATE TABLE IF NOT EXISTS bot_settings (
        key TEXT PRIMARY KEY,
        value TEXT
    )
'''
c.execute('''
    CREATE TABLE IF NOT EXISTS private_combos (
        user_id INTEGER,
        country_code TEXT,
        numbers TEXT,
        PRIMARY KEY (user_id, country_code)
    )
'''
#  جدول القنوات الجديدة
c.execute('''
    CREATE TABLE IF NOT EXISTS force_sub_channels (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        channel_url TEXT UNIQUE NOT NULL,

```



```

        description TEXT DEFAULT '',
        enabled INTEGER DEFAULT 1
    )
'''

# تهيئة الإعدادات القديمة (للتوافق مع البوت القديم)
c.execute("INSERT OR IGNORE INTO bot_settings (key, value) VALUES ('force_sub
c.execute("INSERT OR IGNORE INTO bot_settings (key, value) VALUES ('force_sub

# 🔄 نقل القناة القديمة (إن وُجدت) تلقائيًا إلى الجدول الجديد
c.execute("SELECT value FROM bot_settings WHERE key = 'force_sub_channel'")
old_channel = c.fetchone()
if old_channel and old_channel[0].strip():
    channel = old_channel[0].strip()
    # تأكد أنها ليست مكررة في الجدول الجديد
    c.execute("SELECT 1 FROM force_sub_channels WHERE channel_url = ?", (chan
if not c.fetchone():
    enabled = 1 if get_setting("force_sub_enabled") == "1" else 0
    c.execute("INSERT INTO force_sub_channels (channel_url, description,
               (channel, "القناة الأساسية", enabled))

conn.commit()
conn.close()

init_db()

# PocketMoneyHub | دوال إدارة قاعدة البيانات

def get_user(user_id):
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute("SELECT * FROM users WHERE user_id = ?", (user_id,))
    row = c.fetchone()
    conn.close()
    return row

def save_user(user_id, username="", first_name="", last_name="", country_code=Non
    """
    (INSERT OR REPLACE). يحفظ أو يحدّث بيانات المستخدم باستخدام استعلام واحد
    في البيئات متعددة الخيوط (race conditions) هذا يمنع أخطاء التزامن
    """
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()

    # نحتاج إلى جلب البيانات القديمة التي لا نريد تغييرها إذا لم يتم توفيرها
    # الدالة بمعلومات أساسية فقط country_code هذا يمنع مسح البيانات القيمة مثل
    existing_data = get_user(user_id)

```

```

if existing_data:
    # جديد، استخدم القديم country_code إذا لم يتم توفير
    if country_code is None:
        country_code = existing_data[4]
    # جديد، استخدم القديم assigned_number إذا لم يتم توفير
    if assigned_number is None:
        assigned_number = existing_data[5]
    # جديد، استخدم القديم private_combo_country إذا لم يتم توفير
    if private_combo_country is None:
        private_combo_country = existing_data[7]

c.execute("""
    REPLACE INTO users (user_id, username, first_name, last_name, country_cod
    VALUES (?, ?, ?, ?, ?, ?, COALESCE((SELECT is_banned FROM users WHERE use
""", (
    user_id,
    username,
    first_name,
    last_name,
    country_code,
    assigned_number,
    user_id, # ل جلب حالة الحظر القديمة COALESCE يُستخدم في
    private_combo_country
))
conn.commit()
conn.close()

def ban_user(user_id):
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute("UPDATE users SET is_banned=1 WHERE user_id=?", (user_id,))
    conn.commit()
    conn.close()

def unban_user(user_id):
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute("UPDATE users SET is_banned=0 WHERE user_id=?", (user_id,))
    conn.commit()
    conn.close()

def is_banned(user_id):
    user = get_user(user_id)
    return user and user[6] == 1

def is_maintenance_mode():
    return not BOT_ACTIVE

```

```

def set_maintenance_mode(status):
    global BOT_ACTIVE
    BOT_ACTIVE = not status

def get_all_users():
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute("SELECT user_id FROM users WHERE is_banned=0")
    users = [row[0] for row in c.fetchall()]
    conn.close()
    return users

def get_combo(country_code, combo_index=1, user_id=None):
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    if user_id:
        c.execute("SELECT numbers FROM private_combos WHERE user_id=? AND country_code=?")
        row = c.fetchone()
        if row:
            conn.close()
            return json.loads(row[0])
        c.execute("SELECT numbers FROM combos WHERE country_code=? AND combo_index=?"
        row = c.fetchone()
        conn.close()
        return json.loads(row[0]) if row else []

def save_combo(country_code, numbers, user_id=None):
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()

    if user_id:
        c.execute("REPLACE INTO private_combos (user_id, country_code, numbers) VALUES
        (user_id, country_code, json.dumps(numbers))")
    else:
        # لهذه الدولة combo_index البحث عن آخر
        c.execute("SELECT MAX(combo_index) FROM combos WHERE country_code=?", (country_code,))
        max_index = c.fetchone()[0]
        next_index = 1 if max_index is None else max_index + 1

        c.execute("INSERT INTO combos (country_code, combo_index, numbers) VALUES
        (country_code, next_index, json.dumps(numbers))")

    conn.commit()
    conn.close()

def delete_combo(country_code, combo_index=None, user_id=None):

```

```

"""
دالة حذف كومبو مع معالجة أخطاء قاعدة البيانات
"""

conn = None
try:
    # ⚠ استخدم timeout كبير و check_same_thread=False
    conn = sqlite3.connect(DB_PATH, timeout=30.0, check_same_thread=False)
    c = conn.cursor()

    if user_id:
        c.execute("DELETE FROM private_combos WHERE user_id=? AND country_cod
    elif combo_index:
        c.execute("DELETE FROM combos WHERE country_code=? AND combo_index=?"
    else:
        c.execute("DELETE FROM combos WHERE country_code=?", (country_code,))

    conn.commit()
    print(f"✅ تم حذف كومبو: {country_code} (index: {combo_index})")
    return True

except sqlite3.Error as e:
    print(f"❌ خطأ SQLite في delete_combo: {e}")
    if conn:
        conn.rollback()
    return False
finally:
    if conn:
        conn.close()

def get_all_combos():
    """(country_code, combo_index) ترجع قائمة من"""
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute("SELECT country_code, combo_index FROM combos ORDER BY country_code
    combos = c.fetchall()
    conn.close()
    return combos # [(country_code, combo_index), ...]

def assign_number_to_user(user_id, number):
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute("UPDATE users SET assigned_number=? WHERE user_id=?", (number, user
    conn.commit()
    conn.close()

def get_user_by_number(number):
    conn = sqlite3.connect(DB_PATH)

```

```

        c = conn.cursor()
        c.execute("SELECT user_id FROM users WHERE assigned_number=?", (number,))
        row = c.fetchone()
        conn.close()
        return row[0] if row else None

def log_otp(number, otp, full_message, assigned_to=None):
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute("INSERT INTO otp_logs (number, otp, full_message, timestamp, assign
                (number, otp, full_message, datetime.now().strftime("%Y-%m-%d %H:%M
    conn.commit()
    conn.close()

def release_number(old_number):
    if not old_number:
        return
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute("UPDATE users SET assigned_number=NULL WHERE assigned_number=?", (o
    conn.commit()
    conn.close()

def get_otp_logs():
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute("SELECT * FROM otp_logs")
    logs = c.fetchall()
    conn.close()
    return logs

def get_user_info(user_id):
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute("SELECT * FROM users WHERE user_id=?", (user_id,))
    row = c.fetchone()
    conn.close()
    return row

# --- دوال إدارة قنوات الاشتراك الإجباري (متعددة) ---
def get_all_force_sub_channels(enabled_only=True):
    """ جلب القنوات (المفعلة فقط أو جميعها) """
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    if enabled_only:
        c.execute("SELECT id, channel_url, description FROM force_sub_channels WH
    else:

```

```

        c.execute("SELECT id, channel_url, description FROM force_sub_channels OR
rows = c.fetchall()
conn.close()
return rows

def add_force_sub_channel(channel_url, description=""):
    """إضافة قناة جديدة (لا تسمح بالتكرار)"""
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    try:
        c.execute("INSERT INTO force_sub_channels (channel_url, description, enab
                    (channel_url.strip(), description.strip()))
        conn.commit()
        return True
    except sqlite3.IntegrityError:
        return False # قناة مكررة
    finally:
        conn.close()

def delete_force_sub_channel(channel_id):
    """حذف قناة بالرقم التعريفي"""
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute("DELETE FROM force_sub_channels WHERE id = ?", (channel_id,))
    changed = c.rowcount > 0
    conn.commit()
    conn.close()
    return changed

def toggle_force_sub_channel(channel_id):
    """تفعيل/تعطيل قناة"""
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute("UPDATE force_sub_channels SET enabled = 1 - enabled WHERE id = ?",
    conn.commit()
    conn.close()

# PocketMoneyHub | الاشتراك الإجباري
def force_sub_check(user_id):
    """التحقق من اشتراك المستخدم في **جميع** القنوات المفصلة"""
    channels = get_all_force_sub_channels(enabled_only=True)
    if not channels:
        return True # لا توجد قنوات → لا يوجد تحقق

    for _, url, _ in channels:
        try:
            # توحيد التنسيق: @xxx بدل https://t.me/xxx

```

```

        if url.startswith("https://t.me/"):
            ch = "@" + url.split("/")[-1]
        elif url.startswith("@"):
            ch = url
        else:
            continue # تجاهل الروابط غير الصحيحة
        member = bot.get_chat_member(ch, user_id)
        if member.status not in ["member", "administrator", "creator"]:
            return False
    except Exception as e:
        print(f"[!] خطأ في التحقق من القناة {url}: {e}")
        return False # أي فشل = غير مشترك
return True

def force_sub_markup():
    """إنشاء زر لكل قناة مُفعّلة + زر التحقق"""
    channels = get_all_force_sub_channels(enabled_only=True)
    if not channels:
        return None

    markup = types.InlineKeyboardMarkup()
    for _, url, desc in channels:
        text = f"🔊 {desc}" if desc else "🔊 القناة في الاشتراك"
        markup.add(types.InlineKeyboardButton(text, url=url))
    markup.add(types.InlineKeyboardButton("✅ تحقق من الاشتراك", callback_data="ch"))
    return markup

# PocketMoneyHub | إنشاء بوت Telegram
bot = telebot.TeleBot(BOT_TOKEN)

# PocketMoneyHub | وظائف البوت التفاعلي
def is_admin(user_id):
    return user_id in ADMIN_IDS

def safe_html(text):
    """غير الصالحة HTML تقوم بتنظيف النص من علامات"""
    if not text:
        return ""
    # بدائل آمنة HTML استبدال علامات
    text = str(text)
    text = text.replace('&', '&')
    text = text.replace('<', '<')
    text = text.replace('>', '>')
    text = text.replace('"', '"')
    return text

@bot.message_handler(commands=['start'])

```

```

def send_welcome(message):
    try:
        user_id = message.from_user.id
        chat_id = message.chat.id

        print(f"[DEBUG] /start command received from user {user_id}")

        # 1. مع صورة (Maintenance Mode) فحص وضع الصيانة
        if is_maintenance_mode() and not is_admin(user_id):
            maintenance_caption = (
                "<b>○ — <u>WELCOME TO 24XRAVEN BOT</u> — ○</b>\n\n"
                "<b>⚠ عذراً عزيزي المستخدم..</b>\n"
                "<b>البوت الآن في وضع الصيانة لتحديث الخدمات.</b>\n\n"
                "<b>⌚ يرجى المحاولة مرة أخرى لاحقاً.</b>\n"
                "<b>—————</b>"
            )
            # file_id استبدل الرابط أدناه برابط صورتك الخاصة أو
            maintenance_photo = "https://imgbly.com/ib/XJgaRvGr7IQiR3g_1770759464"

            try:
                bot.send_photo(
                    chat_id,
                    maintenance_photo,
                    caption=maintenance_caption,
                    parse_mode="HTML"
                )
            except:
                # في حال فشل إرسال الصورة نرسل النص فقط كبديل
                bot.send_message(chat_id, maintenance_caption, parse_mode="HTML")
            return

        # 2. فحص الحظر (Banned Users)
        if is_banned(user_id):
            bot.reply_to(message, "<b>🚫 عذراً، لقد تم حظرك من استخدام البوت.</b>"
            return

        # 3. فحص الاشتراك الإجباري (Force Subscribe)
        if not force_sub_check(user_id):
            markup = force_sub_markup()
            if markup:
                bot.send_message(chat_id, "<b>🔒 الاشتراك في القنوات لاستخدام البوت"
            else:
                bot.send_message(chat_id, "<b>🔒 جباري مفعل لكن لم يتم تحديد قناة"
            return

        # 4. حفظ المستخدم الجديد وإشعار الإدارة
        if not get_user(user_id):

```



```

save_user(
    user_id,
    username=message.from_user.username or "",
    first_name=message.from_user.first_name or "",
    last_name=message.from_user.last_name or ""
)
for admin in ADMIN_IDS:
    try:
        caption = (
            f"🆕 <b>مستخدم جديد دخل البوت:</b>\n"
            f"<b>ID:</b> <code>{user_id}</code>\n"
            f"<b>👤:</b> @{safe_html(message.from_user.username or 'N"
            f"<b>الاسم:</b> {safe_html(message.from_user.first_name or
        )
        bot.send_message(admin, caption, parse_mode="HTML")
    except:
        pass

# 5. بناء قائمة الأزرار (الدول والكومبوهات)
markup = types.InlineKeyboardMarkup(row_width=2)
buttons = []
user_data = get_user(user_id)
private_combo = user_data[7] if user_data else None
all_combos = get_all_combos()

# تجميع الكومبوهات لكل دولة
country_combos = {}
for country_code, combo_index in all_combos:
    if country_code not in country_combos:
        country_combos[country_code] = []
    country_combos[country_code].append(combo_index)

# الكومبو الخاص أولاً
if private_combo and private_combo in COUNTRY_CODES:
    name, flag, _ = COUNTRY_CODES[private_combo]
    buttons.append(types.InlineKeyboardButton(f"{flag} {name} (Private)",

# عمل أزرار لكل كومبو
for country_code, indices in country_combos.items():
    if country_code in COUNTRY_CODES and country_code != private_combo:
        name, flag, _ = COUNTRY_CODES[country_code]
        for idx in indices:
            # إذا كان الكومبو الأول فقط أو دولة واحدة فقط، ما نضيف رقم
            if len(indices) == 1:
                btn_text = f"{flag} {name}"
            else:
                btn_text = f"{flag} {name} ({idx})"

```

```

        buttons.append(types.InlineKeyboardButton(btn_text, callback_)

for i in range(0, len(buttons), 2):
    markup.row(*buttons[i:i+2])

# زر لوحة التحكم للأدمن فقط
if is_admin(user_id):
    markup.add(types.InlineKeyboardButton("🔒 Admin Panel", callback_data

# 6. الرسالة الترحيبية المنسقة
fancy_text = (
    "<b>O<u>WELCOME TO 24XRAVEN BOT </u>O</b>\n\n"
    "<b>🇸🇦 <u>Fast • Secure • online</u></b>\n\n"
    "<b>🎓 <u>Owner</u> • <a href='https://t.me/nolab420'>Nolab</a></b>\n"
    "<b>—————</b>\n"
    "<b><u>اختر الدولة التي تريدها من الزر الاسفل</u> ⬇️</b>"
)

bot.send_message(
    chat_id,
    fancy_text,
    parse_mode="HTML",
    reply_markup=markup,
    disable_web_page_preview=True
)

print(f"[DEBUG] Welcome message sent successfully to user {user_id}")
except Exception as e:
    print(f"❌ Error in send_welcome: {e}")
    traceback.print_exc()
    try:
        bot.send_message(message.chat.id, "❌ حدث خطأ، جرب مرة أخرى /start")
    except:
        pass

@bot.callback_query_handler(func=lambda call: call.data == "check_sub")
def check_subscription(call):
    if force_sub_check(call.from_user.id):
        bot.answer_callback_query(call.id, "✅ نعم التحقق! يمكنك استخدام البوت الآن", show_alert=True)
        send_welcome(call.message)
    else:
        bot.answer_callback_query(call.id, "❌ لم تشترك بعد!", show_alert=True)

@bot.callback_query_handler(func=lambda call: call.data.startswith("country_"))
def handle_country_selection(call):
    try:
        user_id = call.from_user.id
        chat_id = call.message.chat.id

```

```

message_id = call.message.message_id

# 1. الفحوصات الأمنية (حظر واشتراك)
if is_banned(user_id):
    bot.answer_callback_query(call.id, "🚫 You are banned.", show_alert=True)
    return
if not force_sub_check(user_id):
    markup = force_sub_markup()
    bot.send_message(chat_id, "<b>🔒 يجب الاشتراك في القناة لاستخدام البوت")
    return

# 2. combo_index استخراج الدولة و
parts = call.data.split("_")
country_code = parts[1]
combo_index = int(parts[2]) if len(parts) > 2 else 1

available_numbers = get_available_numbers(country_code, combo_index, user_id)

if not available_numbers:
    # رسالة خطأ فخمة في حال عدم توفر أرقام
    error_msg = "<b>❌ نعتذر، جميع الأرقام قيد الاستخدام حالياً لهذه الدولة"
    markup = types.InlineKeyboardMarkup()
    markup.add(types.InlineKeyboardButton("⬅️ BACK العودة لاختيار دولة أخرى", callback_data="back"))
    bot.edit_message_text(error_msg, chat_id, message_id, reply_markup=markup)
    return

# 3. تخصيص الرقم وتحرير القديم
assigned = random.choice(available_numbers)
old_user = get_user(user_id)
if old_user and old_user[5]:
    release_number(old_user[5])

assign_number_to_user(user_id, assigned)
save_user(user_id, country_code=country_code, assigned_number=assigned)

# 4. جلب بيانات الدولة وتنسيق النص
name, flag, _ = COUNTRY_CODES.get(country_code, ("Unknown", "🌐", ""))
msg_text = (
    f"<b>◆ Number:</b> <code>+{assigned}</code>\n"
    f"<b>◆ Country:</b> {flag} {name}\n"
    f"<b>◆ Combo:</b> #{combo_index}\n"
    f"<b>◆ Status :</b> ⌚ Waiting for SMS"
)

# 5. بناء لوحة الأزرار الفخمة
markup = types.InlineKeyboardMarkup()

```

```

# زر الجروب في الأعلى
markup.add(types.InlineKeyboardButton("View OTP 📞", url="https://t.me/+O

# أزرار التحكم في صف واحد
markup.row(
    types.InlineKeyboardButton("🔄 Change Number", callback_data=f"change
    types.InlineKeyboardButton("⬅️ Back", callback_data="back_to_countrie
)

# 6. التحديث النهائي للرسالة
bot.edit_message_text(
    text=msg_text,
    chat_id=chat_id,
    message_id=message_id,
    reply_markup=markup,
    parse_mode="HTML",
    disable_web_page_preview=True
)
bot.answer_callback_query(call.id, "✅ تم استلام الرقم بنجاح")
except Exception as e:
    print(f"❌ Error in handle_country_selection: {e}")
    traceback.print_exc()
    try:
        bot.answer_callback_query(call.id, "❌ جرب مرة أخرى", show_a
    except:
        pass

@bot.callback_query_handler(func=lambda call: call.data.startswith("change_num_"))
def change_number(call):
    user_id = call.from_user.id

    # 1. الفحوصات الأمنية
    if is_banned(user_id):
        return
    if not force_sub_check(user_id):
        return

    # 2. combo_index استخراج كود الدولة و
    parts = call.data.split("_")
    country_code = parts[2]
    combo_index = int(parts[3]) if len(parts) > 3 else 1

    available_numbers = get_available_numbers(country_code, combo_index, user_id)

    if not available_numbers:
        bot.answer_callback_query(call.id, "❌ جميع الأرقام قيد الاستخدام حالياً", show_a
        return

```

```

# 3. تحرير الرقم القديم وتعيين الجديد
old_user = get_user(user_id)
if old_user and old_user[5]:
    release_number(old_user[5])

assigned = random.choice(available_numbers)
assign_number_to_user(user_id, assigned)
save_user(user_id, assigned_number=assigned)

# 4. جلب بيانات الدولة والتنسيق الفخم
name, flag, _ = COUNTRY_CODES.get(country_code, ("Unknown", "🌐", ""))

msg_text = (
    f"<b>◆ Number:</b> <code>+{assigned}</code>\n"
    f"<b>◆ Country:</b> {flag} {name}\n"
    f"<b>◆ Combo:</b> #{combo_index}\n"
    f"<b>◆ Status :</b> ⌚ Waiting for SMS"
)

# 5. بناء الأزرار المحدثة
markup = types.InlineKeyboardMarkup()

# زر الجروب (استبدل الرابط برابط جروبك الحقيقي)
markup.add(types.InlineKeyboardButton("View OTP 🙄", url="https://t.me/+OQ0CT..."))

# أزرار التحكم
markup.row(
    types.InlineKeyboardButton("🔄 Change Number", callback_data=f"change_nur..."),
    types.InlineKeyboardButton("⬅️ BACK Back", callback_data="back_to_countries")
)

# 6. تحديث الرسالة
try:
    bot.edit_message_text(
        text=msg_text,
        chat_id=call.message.chat.id,
        message_id=call.message.message_id,
        reply_markup=markup,
        parse_mode="HTML",
        disable_web_page_preview=True
    )
    # إشعار سريع بنجاح التغيير
    bot.answer_callback_query(call.id, "✅ تم تغيير الرقم بنجاح")
except Exception as e:
    print(f"Error in change_number: {e}")
    bot.answer_callback_query(call.id)

```

```

@bot.callback_query_handler(func=lambda call: call.data == "back_to_countries")
def back_to_countries(call):
    # 1. بناء قائمة الأزرار
    markup = types.InlineKeyboardMarkup(row_width=2)
    buttons = []

    # جلب البيانات
    user = get_user(call.from_user.id)
    private_combo = user[7] if user else None
    all_combos = get_all_combos()

    # تجميع الكومبوهات لكل دولة
    country_combos = {}
    for country_code, combo_index in all_combos:
        if country_code not in country_combos:
            country_combos[country_code] = []
        country_combos[country_code].append(combo_index)

    # إضافة الكومبو الخاص أولاً (إذا وُجد)
    if private_combo and private_combo in COUNTRY_CODES:
        name, flag, _ = COUNTRY_CODES[private_combo]
        buttons.append(types.InlineKeyboardButton(f"{flag} {name} (Private)", cal

    # إضافة الكومبوهات العامة
    for country_code, indices in country_combos.items():
        if country_code in COUNTRY_CODES and country_code != private_combo:
            name, flag, _ = COUNTRY_CODES[country_code]
            for idx in indices:
                # إذا كان الكومبو الأول فقط أو دولة واحدة فقط، ما نضيف رقم
                if len(indices) == 1:
                    btn_text = f"{flag} {name}"
                else:
                    btn_text = f"{flag} {name} ({idx})"
                buttons.append(types.InlineKeyboardButton(btn_text, callback_data

    # توزيع الأزرار في صفوف
    for i in range(0, len(buttons), 2):
        markup.row(*buttons[i:i+2])

    # إضافة زر الإدارة للمشرفين
    if is_admin(call.from_user.id):
        admin_btn = types.InlineKeyboardButton("🔑 Admin Panel", callback_data="a
        markup.add(admin_btn)

    # 2. النص المنسق (نفس تنسيق رسالة الترحيب)
    fancy_text = (

```

```

"<b>O<u>WELCOME TO 24XRAVEN BOT </u>O</b>\n\n"
"<b>🔒 <u>Fast • Secure • online</u></b>\n\n"
"<b>🎓 <u>Owner</u> • <a href='https://t.me/nolab420'>Nolab</a></b>\n\n"
"<b>_____</b>\n"
"<b><u>اختر الدولة التي تريدها من الزر الاسفل </u> ⬇️</b>"
)

```

3. تعديل الرسالة الحالية

```

try:
    bot.edit_message_text(
        chat_id=call.message.chat.id,
        message_id=call.message.message_id,
        text=fancy_text,
        parse_mode="HTML",
        reply_markup=markup,
        disable_web_page_preview=True
    )
except Exception as e:
    print(f"Error editing message: {e}")
    bot.answer_callback_query(call.id)

```

PocketMoneyHub | لوحة التحكم الإدارية

user_states = {}

def admin_main_menu():

markup = types.InlineKeyboardMarkup()

1. زر حالة البوت (يحتل الصدارة)

status_icon = "🟢" if not is_maintenance_mode() else "🔴"

status_text = "الآن: يعمل بنجاح" if not is_maintenance_mode() else "يد الصيانة"

markup.add(types.InlineKeyboardButton(f"{status_icon} {status_text} {status_i

2. قسم إدارة الكومبوهات (أزرار كبيرة)

markup.row(

types.InlineKeyboardButton("📁 إضافة كومبو", callback_data="admin_add_com

types.InlineKeyboardButton("🗑️ حذف كومبو", callback_data="admin_del_combc

)

3. قسم الإحصائيات والتقارير

markup.row(

types.InlineKeyboardButton("📊 الإحصائيات", callback_data="admin_stats"),

types.InlineKeyboardButton("📄 تقرير شامل", callback_data="admin_full_rep

)

4. قسم الإذاعة (Broadcast)

markup.row(

types.InlineKeyboardButton("📢 إذاعة عامة", callback_data="admin_broadcas

```

        types.InlineKeyboardButton("📢 إذاعة مخصصة", callback_data="admin_broadcast")
    )

# 5. قسم إدارة المستخدمين
markup.row(
    types.InlineKeyboardButton("🚫 حظر", callback_data="admin_ban"),
    types.InlineKeyboardButton("✅ إلغاء حظر", callback_data="admin_unban"),
    types.InlineKeyboardButton("👤 معلومات", callback_data="admin_user_info")
)

# 6. قسم الإعدادات المتقدمة
markup.row(
    types.InlineKeyboardButton("🔗 اشتراك", callback_data="admin_force_sub"),
    types.InlineKeyboardButton("🖥️ اللوحات", callback_data="admin_dashboards"),
    types.InlineKeyboardButton("🔑 برايفت", callback_data="admin_private_comb")
)

# 7. زر الخروج
markup.add(types.InlineKeyboardButton("⬅️ BACK مغادرة لوحة التحكم", callback_data=

return markup

@bot.callback_query_handler(func=lambda call: call.data == "admin_panel")
def show_admin_panel(call):
    # التحقق من الرتبة أولاً
    if not is_admin(call.from_user.id):
        bot.answer_callback_query(call.id, "⚠️ عذراً، هذا القسم للمطورين فقط", sh
        return

# النص المنسق فخم جداً
admin_text = (
    "<b>○—— <u>LOGIN ADMIN PANEL</u> ——○</b>\n\n"
    "<b>👋 مرحباً بك يا مطور في لوحة التحكم.</b>\n\n"
    "<b>⚙️ يمكنك التحكم في كامل وظائف البوت من هنا.</b>\n"
    "<b>⚠️ تنبيه: أي تغيير في الإعدادات يؤثر على المستخدمين فوراً.</b>\n\n"
    "<b>—————</b>\n"
    "<b>إحصائيات سريعة:</b>\n"
    "<b>• حالة السيرفر <u>Online</u> ✅</b>\n"
    f"<b>• الوقت الحالي: <u>{datetime.now().strftime('%H:%M')}</u></b>\n"
    "<b>—————</b>"
)

try:
    bot.edit_message_text(
        chat_id=call.message.chat.id,
        message_id=call.message.message_id,
        text=admin_text,

```



```

        parse_mode="HTML",
        reply_markup=admin_main_menu(),
        disable_web_page_preview=True
    )
except Exception as e:
    print(f"Admin Panel Error: {e}")

# PocketMoneyHub | ميزة الاشتراك الإجباري في لوحة الإدارة
@bot.callback_query_handler(func=lambda call: call.data == "admin_force_sub")
def admin_force_sub(call):
    if not is_admin(call.from_user.id):
        return

    channels = get_all_force_sub_channels(enabled_only=False)
    text = "🔒: إدارة قنوات الاشتراك الإجباري\n"
    text += f"إجمالي القنوات: {len(channels)}\n"
    text += "_____ \n"

    markup = types.InlineKeyboardMarkup()
    for ch_id, url, desc in channels:
        # جلب الحالة بدقة
        conn = sqlite3.connect(DB_PATH)
        c = conn.cursor()
        c.execute("SELECT enabled FROM force_sub_channels WHERE id=?", (ch_id,))
        enabled = c.fetchone()[0]
        conn.close()
        status = "✅" if enabled else "❌"
        btn_text = f"{status} {desc or url[:25]}"
        markup.add(types.InlineKeyboardButton(btn_text, callback_data=f"edit_forc

    markup.add(types.InlineKeyboardButton("➕ إضافة قناة", callback_data="add_for
    markup.add(types.InlineKeyboardButton("⬅️ Back", callback_data="admin_panel")
    bot.edit_message_text(text, call.message.chat.id, call.message.message_id, re

@bot.callback_query_handler(func=lambda call: call.data == "toggle_maintenance")
def handle_maintenance_toggle(call):
    if not is_admin(call.from_user.id): return

    # عكس الحالة الحالية
    current_status = is_maintenance_mode()
    set_maintenance_mode(not current_status) # دالة الحفظ

    new_status_text = "🔒 تم فتح البوت للجميع" if current_status else "🔓 صيانة"

    # إشعار سريع للأدمن
    bot.answer_callback_query(call.id, new_status_text, show_alert=True)

```

```

# تحديث اللوحة فوراً ليتغير شكل الزر
bot.edit_message_reply_markup(call.message.chat.id, call.message.message_id,

# --- إضافة قناة جديدة ---
@bot.callback_query_handler(func=lambda call: call.data == "add_force_ch")
def add_force_ch_step1(call):
    if not is_admin(call.from_user.id):
        return
    user_states[call.from_user.id] = "add_force_ch_url"
    markup = types.InlineKeyboardMarkup()
    markup.add(types.InlineKeyboardButton("⬅️ Back", callback_data="admin_force_s
    bot.edit_message_text("أرسل رابط القناة (مثل: https://t.me/xxx أو @xxx):", ca

@bot.message_handler(func=lambda msg: user_states.get(msg.from_user.id) == "add_f
def add_force_ch_step2(message):
    url = message.text.strip()
    if not (url.startswith("@") or url.startswith("https://t.me/")):
        bot.reply_to(message, "❌ رابط غير صالح! يجب أن يبدأ بـ @ أو https://t.me
        return
    user_states[message.from_user.id] = {"step": "add_force_ch_desc", "url": url}
    bot.reply_to(message, "أدخل وصفاً للقناة (أو اترك فارغاً)")

@bot.message_handler(func=lambda msg: isinstance(user_states.get(msg.from_user.id
def add_force_ch_step3(message):
    data = user_states[message.from_user.id]
    url = data["url"]
    desc = message.text.strip()
    if add_force_sub_channel(url, desc):
        bot.reply_to(message, f"✅ تم إضافة القناة \n{url}\nالوصف: {desc or '-'}"
    else:
        bot.reply_to(message, "❌ القناة موجودة مسبقاً")
    del user_states[message.from_user.id]

# --- تعديل/حذف قناة فردية ---
@bot.callback_query_handler(func=lambda call: call.data.startswith("edit_force_ch
def edit_force_ch(call):
    if not is_admin(call.from_user.id):
        return
    try:
        ch_id = int(call.data.split("_", 3)[3])
    except:
        return
    # جلب بيانات القناة
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute("SELECT channel_url, description, enabled FROM force_sub_channels W
    row = c.fetchone()

```

```

conn.close()
if not row:
    bot.answer_callback_query(call.id, "❌ القناة غير موجودة", show_alert=True)
    return

url, desc, enabled = row
status = "مفعلة" if enabled else "معطلة"
text = f"🔧 إدارة القناة: الرابط: {url}\nالوصف: {desc or '-'}\nالحالة: {status}"
markup = types.InlineKeyboardMarkup()
markup.add(types.InlineKeyboardButton("✎ تعديل الوصف", callback_data=f"edit_desc_{ch_id}"))
if enabled:
    markup.add(types.InlineKeyboardButton("❌ تعطيل", callback_data=f"toggle_ch_{ch_id}"))
else:
    markup.add(types.InlineKeyboardButton("✅ تفعيل", callback_data=f"toggle_ch_{ch_id}"))
markup.add(types.InlineKeyboardButton("🗑 حذف", callback_data=f"del_ch_{ch_id}"))
markup.add(types.InlineKeyboardButton("⬅ BACK", callback_data="admin_force_sub"))
bot.edit_message_text(text, call.message.chat.id, call.message.message_id, reply_markup=markup)

@bot.callback_query_handler(func=lambda call: call.data.startswith("toggle_ch_"))
def toggle_ch(call):
    ch_id = int(call.data.split("_", 2)[2])
    toggle_force_sub_channel(ch_id)
    bot.answer_callback_query(call.id, "🔄 تم تغيير حالة القناة", show_alert=True)
    admin_force_sub(call)

@bot.callback_query_handler(func=lambda call: call.data.startswith("del_ch_"))
def del_ch(call):
    ch_id = int(call.data.split("_", 2)[2])
    if delete_force_sub_channel(ch_id):
        bot.answer_callback_query(call.id, "✅ الحذف", show_alert=True)
    else:
        bot.answer_callback_query(call.id, "❌ فشل الحذف", show_alert=True)
    admin_force_sub(call)

@bot.callback_query_handler(func=lambda call: call.data.startswith("edit_desc_"))
def edit_desc_step1(call):
    ch_id = int(call.data.split("_", 2)[2])
    user_states[call.from_user.id] = f"edit_desc_{ch_id}"
    markup = types.InlineKeyboardMarkup()
    markup.add(types.InlineKeyboardButton("⬅ BACK", callback_data=f"edit_force_sub_{ch_id}"))
    bot.edit_message_text("أدخل الوصف الجديد:", call.message.chat.id, call.message.message_id, reply_markup=markup)

@bot.message_handler(func=lambda msg: isinstance(msg, User))
def edit_desc_step2(message):
    try:
        ch_id = int(user_states[message.from_user.id].split("_", 2)[2])
        desc = message.text.strip()
    except:
        pass

```

```

    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute("UPDATE force_sub_channels SET description = ? WHERE id = ?", (
    conn.commit()
    conn.close()
    bot.reply_to(message, "✅ تم تحديث الوصف!")
except Exception as e:
    bot.reply_to(message, f"❌ خطأ: {e}")
del user_states[message.from_user.id]

@bot.callback_query_handler(func=lambda call: call.data == "admin_add_combo")
def admin_add_combo(call):
    if not is_admin(call.from_user.id):
        return
    user_states[call.from_user.id] = "waiting_combo_file"
    markup = types.InlineKeyboardMarkup()
    markup.add(types.InlineKeyboardButton("⬅️ Back", callback_data="admin_panel"))
    bot.edit_message_text("📎 أرسل ملف الكومبو بصيغة TXT", call.message.chat.id,

@bot.message_handler(content_types=['document'])
def handle_combo_file(message):
    if not is_admin(message.from_user.id):
        return
    if user_states.get(message.from_user.id) != "waiting_combo_file":
        return
    try:
        file_info = bot.get_file(message.document.file_id)
        downloaded_file = bot.download_file(file_info.file_path)
        content = downloaded_file.decode('utf-8')
        lines = [line.strip() for line in content.splitlines() if line.strip()]
        if not lines:
            bot.reply_to(message, "❌ الملف فارغ!")
            return
        first_num = clean_number(lines[0])
        country_code = None
        for code in COUNTRY_CODES:
            if first_num.startswith(code):
                country_code = code
                break
        if not country_code:
            bot.reply_to(message, "❌ لا يمكن تحديد الدولة من الأرقام")
            return
        save_combo(country_code, lines)
        name, flag, _ = COUNTRY_CODES[country_code]
        bot.reply_to(message, f"✅ تم حفظ الكومبو لدولة {flag} {name}\n📄 الأرقام")
        del user_states[message.from_user.id]
    except Exception as e:

```

```

bot.reply_to(message, f"❌ خطأ: {e}")

@bot.callback_query_handler(func=lambda call: call.data == "admin_del_combo")
def admin_del_combo(call):
    if not is_admin(call.from_user.id):
        return
    combos = get_all_combos()
    if not combos:
        bot.answer_callback_query(call.id, "لا توجد كومبوهات")
        return
    markup = types.InlineKeyboardMarkup()
    # تجميع الكومبوهات لكل دولة
    country_combos = {}
    for country_code, combo_index in combos:
        if country_code not in country_combos:
            country_combos[country_code] = []
        country_combos[country_code].append(combo_index)

    for country_code, indices in country_combos.items():
        if country_code in COUNTRY_CODES:
            name, flag, _ = COUNTRY_CODES[country_code]
            for idx in indices:
                # إذا كان الكومبو الأول فقط أو دولة واحدة فقط، ما نضيف رقم
                if len(indices) == 1:
                    btn_text = f"{flag} {name}"
                else:
                    btn_text = f"{flag} {name} ({idx})"
                markup.add(types.InlineKeyboardButton(btn_text, callback_data=f"d

    markup.add(types.InlineKeyboardButton("⬅️ Back", callback_data="admin_panel"))
    bot.edit_message_text("اختر الكومبو لحذف:", call.message.chat.id, call.messa

@bot.callback_query_handler(func=lambda call: call.data.startswith("del_combo_"))
def confirm_del_combo(call):
    if not is_admin(call.from_user.id):
        return

    parts = call.data.split("_")
    country_code = parts[2]
    combo_index = int(parts[3]) if len(parts) > 3 else 1

    # استدعاء الدالة المعدلة
    success = delete_combo(country_code, combo_index)

    name, flag, _ = COUNTRY_CODES.get(country_code, ("Unknown", "🌐", ""))

    if success:

```

```

        bot.answer_callback_query(call.id, f"✅ تم حذف الكومبو : {flag} {name} ({c
else:
        bot.answer_callback_query(call.id, f"❌ فشل حذف الكومبو!", show_alert=Tru

# تحديث القائمة
admin_del_combo(call)

@bot.callback_query_handler(func=lambda call: call.data == "admin_stats")
def admin_stats(call):
    if not is_admin(call.from_user.id):
        return
    total_users = len(get_all_users())
    combos = get_all_combos()

    # حساب عدد الكومبوهات الفريدة
    unique_countries = set()
    total_combos = 0
    for country_code, combo_index in combos:
        unique_countries.add(country_code)
        total_combos += 1

    total_numbers = 0
    for country_code, combo_index in combos:
        total_numbers += len(get_combo(country_code, combo_index))

    otp_count = len(get_otp_logs())
    markup = types.InlineKeyboardMarkup()
    markup.add(types.InlineKeyboardButton("⬅️ Back", callback_data="admin_panel"))
    bot.edit_message_text(
        f"🇮🇲 إحصائيات البوت :\n"
        f"👥 المستخدمين النشطين : {total_users}\n"
        f"🌐 الدول المضافة : {len(unique_countries)}\n"
        f"📦 الكومبوهات : {total_combos}\n"
        f"📞 إجمالي الأرقام : {total_numbers}\n"
        f"🔑 إجمالي الأكواد المستلمة : {otp_count}",
        call.message.chat.id, call.message.message_id, reply_markup=markup
    )

@bot.callback_query_handler(func=lambda call: call.data == "admin_full_report")
def admin_full_report(call):
    if not is_admin(call.from_user.id):
        return
    try:
        report = "🇮🇲 تقرير شامل عن البوت\n" + "="*40 + "\n\n"
        # المستخدمون
        report += "👥 المستخدمين :\n"
        conn = sqlite3.connect(DB_PATH)

```

```

c = conn.cursor()
c.execute("SELECT * FROM users")
users = c.fetchall()
for u in users:
    status = "محظور" if u[6] else "نشط"
    report += f"ID: {u[0]} | @{u[1] or 'N/A'} | الرقم: {u[5] or 'N/A'} |
report += "\n" + "="*40 + "\n\n"
# الأكواد
report += "🔑 سجل الأكواد:\n"
c.execute("SELECT * FROM otp_logs")
logs = c.fetchall()
for log in logs:
    user_info = get_user_info(log[5]) if log[5] else None
    user_tag = f"@{user_info[1]}" if user_info and user_info[1] else f"ID
report += f"الرقم: {log[1]} | الكود: {log[2]} | المستخدم: {user_tag}

# الكومبوهات
report += "\n" + "="*40 + "\n\n"
report += "📦 الكومبوهات:\n"
c.execute("SELECT country_code, combo_index, LENGTH(numbers) FROM combos")
combos_data = c.fetchall()
for country_code, combo_index, num_length in combos_data:
    name, flag, _ = COUNTRY_CODES.get(country_code, ("Unknown", "🌐", ""))
    num_count = len(json.loads(get_combo(country_code, combo_index)))
    report += f"{flag} {name} ({combo_index}): {num_count} رقم\n"

conn.close()
report += "\n" + "="*40 + "\n\n"
report += "تم إنشاء التقرير في " + datetime.now().strftime("%Y-%m-%d %H:
with open("bot_report.txt", "w", encoding="utf-8") as f:
    f.write(report)
with open("bot_report.txt", "rb") as f:
    bot.send_document(call.from_user.id, f)
os.remove("bot_report.txt")
bot.answer_callback_query(call.id, "✅ تم إرسال التقرير", show_alert=True)
except Exception as e:
    bot.answer_callback_query(call.id, f"❌ خطأ: {e}", show_alert=True)

@bot.callback_query_handler(func=lambda call: call.data == "admin_ban")
def admin_ban_step1(call):
    if not is_admin(call.from_user.id):
        return
    user_states[call.from_user.id] = "ban_user"
    markup = types.InlineKeyboardMarkup()
    markup.add(types.InlineKeyboardButton("⬅️ Back", callback_data="admin_panel"))
    bot.edit_message_text("أدخل معرف المستخدم لحظره:", call.message.chat.id, call

```

```

@bot.message_handler(func=lambda msg: user_states.get(msg.from_user.id) == "ban_u
def admin_ban_step2(message):
    try:
        uid = int(message.text)
        ban_user(uid)
        bot.reply_to(message, f"✅ تم حظر المستخدم {uid}")
        del user_states[message.from_user.id]
    except:
        bot.reply_to(message, "❌ معرف غير صحيح!")

@bot.callback_query_handler(func=lambda call: call.data == "admin_unban")
def admin_unban_step1(call):
    if not is_admin(call.from_user.id):
        return
    user_states[call.from_user.id] = "unban_user"
    markup = types.InlineKeyboardMarkup()
    markup.add(types.InlineKeyboardButton("⬅️ Back", callback_data="admin_panel"))
    bot.edit_message_text("أدخل معرف المستخدم لفك حظره:", call.message.chat.id, c

@bot.message_handler(func=lambda msg: user_states.get(msg.from_user.id) == "unban
def admin_unban_step2(message):
    try:
        uid = int(message.text)
        unban_user(uid)
        bot.reply_to(message, f"✅ تم فك حظر المستخدم {uid}")
        del user_states[message.from_user.id]
    except:
        bot.reply_to(message, "❌ معرف غير صحيح!")

@bot.callback_query_handler(func=lambda call: call.data == "admin_broadcast_all")
def admin_broadcast_all_step1(call):
    if not is_admin(call.from_user.id):
        return
    user_states[call.from_user.id] = "broadcast_all"
    markup = types.InlineKeyboardMarkup()
    markup.add(types.InlineKeyboardButton("⬅️ Back", callback_data="admin_panel"))
    bot.edit_message_text("أرسل الرسالة للإرسال للجميع:", call.message.chat.id, ca

@bot.message_handler(func=lambda msg: user_states.get(msg.from_user.id) == "broad
def admin_broadcast_all_step2(message):
    users = get_all_users()
    success = 0
    for uid in users:
        try:
            bot.send_message(uid, message.text)
            success += 1
        except:

```



```

        pass
    bot.reply_to(message, f"✅ تم الإرسال إلى {success}/{len(users)} مستخدم")
    del user_states[message.from_user.id]

@bot.callback_query_handler(func=lambda call: call.data == "admin_broadcast_user")
def admin_broadcast_user_step1(call):
    if not is_admin(call.from_user.id):
        return
    user_states[call.from_user.id] = "broadcast_user_id"
    markup = types.InlineKeyboardMarkup()
    markup.add(types.InlineKeyboardButton("⬅️ Back", callback_data="admin_panel"))
    bot.edit_message_text("أدخل معرف المستخدم:", call.message.chat.id, call.messa

@bot.message_handler(func=lambda msg: user_states.get(msg.from_user.id) == "broad
def admin_broadcast_user_step2(message):
    try:
        uid = int(message.text)
        user_states[message.from_user.id] = f"broadcast_msg_{uid}"
        bot.reply_to(message, "أرسل الرسالة:")
    except:
        bot.reply_to(message, "❌ معرف غير صحيح!")

@bot.message_handler(func=lambda msg: user_states.get(msg.from_user.id, "").start
def admin_broadcast_user_step3(message):
    uid = int(user_states[message.from_user.id].split("_")[2])
    try:
        bot.send_message(uid, message.text)
        bot.reply_to(message, f"✅ تم الإرسال للمستخدم {uid}")
    except Exception as e:
        bot.reply_to(message, f"❌ فشل: {e}")
    del user_states[message.from_user.id]

@bot.callback_query_handler(func=lambda call: call.data == "admin_user_info")
def admin_user_info_step1(call):
    if not is_admin(call.from_user.id):
        return
    user_states[call.from_user.id] = "get_user_info"
    markup = types.InlineKeyboardMarkup()
    markup.add(types.InlineKeyboardButton("⬅️ Back", callback_data="admin_panel"))
    bot.edit_message_text("أدخل معرف المستخدم:", call.message.chat.id, call.messa

@bot.message_handler(func=lambda msg: user_states.get(msg.from_user.id) == "get_u
def admin_user_info_step2(message):
    try:
        uid = int(message.text)
        user = get_user_info(uid)
        if not user:

```

```

        bot.reply_to(message, "❌ المستخدم غير موجود")
        return
    status = "محظور" if user[6] else "نشط"
    info = f"👤 معلومات المستخدم:\n"
    info += f"🆔: {user[0]}\n"
    info += f".Username: @{user[1] or 'N/A'}\n"
    info += f"الاسم: {user[2] or ''} {user[3] or ''}\n"
    info += f"الرقم المخصص: {user[5] or 'N/A'}\n"
    info += f"الحالة: {status}"
    bot.reply_to(message, info)
except Exception as e:
    bot.reply_to(message, f"❌ خطأ: {e}")
del user_states[message.from_user.id]

@bot.callback_query_handler(func=lambda call: call.data == "admin_private_combo")
def admin_private_combo(call):
    if not is_admin(call.from_user.id):
        return
    markup = types.InlineKeyboardMarkup()
    markup.add(types.InlineKeyboardButton("➕ إضافة كومبو برايفت", callback_data=
    markup.add(types.InlineKeyboardButton("🗑 مسح كومبو برايفت", callback_data="c
    markup.add(types.InlineKeyboardButton("⬅ BACK Back", callback_data="admin_panel")
    bot.edit_message_text("👤 كومبو برايفت:", call.message.chat.id, call.message.

@bot.callback_query_handler(func=lambda call: call.data == "add_private_combo")
def add_private_combo_step1(call):
    if not is_admin(call.from_user.id):
        return
    user_states[call.from_user.id] = "add_private_user_id"
    markup = types.InlineKeyboardMarkup()
    markup.add(types.InlineKeyboardButton("⬅ BACK Back", callback_data="admin_private
    bot.edit_message_text("أدخل معرف المستخدم:", call.message.chat.id, call.messa

@bot.message_handler(func=lambda msg: user_states.get(msg.from_user.id) == "add_p
def add_private_combo_step2(message):
    try:
        uid = int(message.text)
        user_states[message.from_user.id] = f"add_private_country_{uid}"
        markup = types.InlineKeyboardMarkup(row_width=2)
        buttons = []
        # تجميع الكومبوهات لكل دولة
        all_combos = get_all_combos()
        country_combos = {}
        for country_code, combo_index in all_combos:
            if country_code not in country_combos:
                country_combos[country_code] = []
            country_combos[country_code].append(combo_index)

```

```

for country_code, indices in country_combos.items():
    if country_code in COUNTRY_CODES:
        name, flag, _ = COUNTRY_CODES[country_code]
        for idx in indices:
            if len(indices) == 1:
                btn_text = f"{flag} {name}"
            else:
                btn_text = f"{flag} {name} ({idx})"
            buttons.append(types.InlineKeyboardButton(btn_text, callback_
for i in range(0, len(buttons), 2):
    markup.row(*buttons[i:i+2])
markup.add(types.InlineKeyboardButton("⬅️ Back", callback_data="admin_pri
bot.reply_to(message, "اختر الدولة:", reply_markup=markup)
except:
    bot.reply_to(message, "❌ معرف غير صحيح!")

@bot.callback_query_handler(func=lambda call: call.data.startswith("select_privat
def select_private_combo(call):
    parts = call.data.split("_")
    uid = int(parts[2])
    country_code = parts[3]
    save_user(uid, private_combo_country=country_code)
    name, flag, _ = COUNTRY_CODES[country_code]
    bot.answer_callback_query(call.id, f"✅ تم تعيين كومبو برايفت لـ {uid} - {fla
admin_private_combo(call)

@bot.callback_query_handler(func=lambda call: call.data == "del_private_combo")
def del_private_combo_step1(call):
    if not is_admin(call.from_user.id):
        return
    user_states[call.from_user.id] = "del_private_user_id"
    markup = types.InlineKeyboardMarkup()
    markup.add(types.InlineKeyboardButton("⬅️ Back", callback_data="admin_private
bot.edit_message_text("أدخل معرف المستخدم:", call.message.chat.id, call.messa

@bot.message_handler(func=lambda msg: user_states.get(msg.from_user.id) == "del_p
def del_private_combo_step2(message):
    try:
        uid = int(message.text)
        save_user(uid, private_combo_country=None)
        bot.reply_to(message, f"✅ تم مسح الكومبو البرايفت للمستخدم {uid}")
    except:
        bot.reply_to(message, "❌ معرف غير صحيح!")
    del user_states[message.from_user.id]

```

```

def get_available_numbers(country_code, combo_index=1, user_id=None):
    all_numbers = get_combo(country_code, combo_index, user_id)
    if not all_numbers:
        return []
    conn = sqlite3.connect(DB_PATH)
    c = conn.cursor()
    c.execute("SELECT assigned_number FROM users WHERE assigned_number IS NOT NULL")
    used_numbers = set(row[0] for row in c.fetchall())
    conn.close()
    available = [num for num in all_numbers if num not in used_numbers]
    return available

```

PocketMoneyHub | الدوال الأساسية للتنظيف والمعالجة

```

def clean_html(text):
    if not text:
        return ""
    text = str(text)
    text = re.sub(r'<[^>]+>', '', text)
    text = text.strip()
    return text

def clean_number(number):
    if not number:
        return ""
    number = re.sub(r'\D', '', str(number))
    return number

def get_country_info(number):
    number = number.strip().replace("+", "").replace(" ", "").replace("-", "")

    for code, (name, flag, short) in COUNTRY_CODES.items():
        if number.startswith(code):
            return name, flag, short

    return "Unknown", "🌐", "UN"

def mask_number(number):
    number = number.strip()
    if len(number) > 8:
        return number[:4] + "...." + number[-3:]
    return number

def extract_otp(message):
    patterns = [
        r'(:code|رمز|كود|verification|تحقق|otp|pin)[\s:]+[\d{3,8}](?:[- ]\d{3,4})?',
        r'(\d{3})[- ](\d{3,4})',
        r'\b(\d{4,8})\b',
    ]

```

```

        r'[](\d{3,8})',
    ]
    for pattern in patterns:
        match = re.search(pattern, message, re.IGNORECASE)
        if match:
            if len(match.groups()) > 1:
                return ''.join(match.groups())
            return match.group(1).replace(' ', '').replace('-', '')
    all_numbers = re.findall(r'\d{4,8}', message)
    if all_numbers:
        return all_numbers[0]
    return "N/A"

def detect_service(message):
    message_lower = message.lower()

    # القاموس الأساسي (زي ما هو)
    services = {
        "#WP": ["whatsapp", "واتس", "واتساب"],
        "#FB": ["facebook", "فيسبوك", "fb"],
        "#IG": ["instagram", "انستا", "انستغرام"],
        "#TG": ["telegram", "تلي", "تيليجرام"],
        "#TW": ["twitter", "تويتر", "x"],
        "#GG": ["google", "gmail", "جوجل", "جميل"],
        "#DC": ["discord", "ديسكورد"],
        "#LN": ["line", "لاين"],
        "#VB": ["viber", "فايبر"],
        "#SK": ["skype", "سكايب"],
        "#SC": ["snapchat", "سناپ"],
        "#TT": ["tiktok", "تيك", "تيك توك"],
        "#AMZ": ["amazon", "امازون"],
        "#APL": ["apple", "ابل", "icloud"],
        "#MS": ["microsoft", "مايكروسوفت"],
        "#IN": ["linkedin", "لينكد"],
        "#UB": ["uber", "اوبر"],
        "#AB": ["airbnb", "ايربنب"],
        "#NF": ["netflix", "نتفليكس"],
        "#SP": ["spotify", "سبوتيفاي"],
        "#YT": ["youtube", "يوتيوب"],
        "#GH": ["github", "جيت هاب"],
        "#PT": ["pinterest", "بنتريست"],
        "#PP": ["paypal", "باي بال"],
        "#BK": ["booking", "بوكنج"],
        "#TL": ["tala", "تالا"],
        "#OLX": ["olx", "اوليكس"],
        "#STC": ["stcpay", "stc"],
    }

```

```

# ✅ التحقق الأساسي (زي ما هو)
for service_code, keywords in services.items():
    for keyword in keywords:
        if keyword in message_lower:
            return service_code

# ✅ Fallback رسالة OTP ذكي من صيغة رسالة
if "code" in message_lower or "verification" in message_lower:
    if "telegram" in message_lower:
        return "#TG"
    if "whatsapp" in message_lower:
        return "#WP"
    if "facebook" in message_lower:
        return "#FB"
    if "instagram" in message_lower:
        return "#IG"
    if "google" in message_lower or "gmail" in message_lower:
        return "#GG"
    if "twitter" in message_lower or "x.com" in message_lower:
        return "#TW"

# آخر حل
return "Unknown"

def html_escape(text):
    return (str(text)
            .replace("&", "&amp;")
            .replace("<", "&lt;") # مهم جداً
            .replace(">", "&gt;")
            .replace("'", "&quot;"))

def format_message(date_str, number, sms):
    country_name, country_flag, country_code = get_country_info(number)
    masked_num = mask_number(number)
    otp_code = extract_otp(sms)
    service = detect_service(sms)

    # التنسيق الجديد بالمللي
    message = (
        f" \n"
        f"| {country_flag} {service} {masked_num} \n"
        f" "
    )
    return message

# iVasms دوال الاتصال بلوحة | PocketMoneyHub

```

```

# PocketMoneyHub | دالة تسجيل الدخول إلى iVasms
def login_to_ivasms():
    """والجلسة CSRF token والحصول على iVasms تسجيل الدخول إلى لوحة"""
    try:
        dash = IVASMS_DASHBOARD
        login_url = dash["login_url"]
        base_url = dash["base_url"]
        username = dash["username"]
        password = dash["password"]
        session = dash["session"]

        print(f"[{dash['name']}] محاولة تسجيل الدخول...")

        # CSRF token الحصول على صفحة الدخول لاستخراج
        login_page_resp = session.get(login_url, timeout=30)
        login_page_resp.raise_for_status()

        # من الصفحة _token استخراج
        soup = BeautifulSoup(login_page_resp.text, 'html.parser')
        token_input = soup.find('input', {'name': '_token'})
        csrf_token = token_input['value'] if token_input else None

        # إعداد بيانات الدخول
        login_data = {
            'email': username,
            'password': password
        }
        if csrf_token:
            login_data['_token'] = csrf_token

        # إرسال طلب الدخول
        login_resp = session.post(login_url, data=login_data, timeout=30)

        # التحقق من نجاح الدخول (إذا تم توجيهنا بعيداً عن صفحة الدخول)
        if "login" not in login_resp.url.lower():
            print(f"[{dash['name']}] ✅ تسجيل الدخول ناجح")

            # جديد من الصفحة بعد الدخول CSRF token استخراج
            dashboard_soup = BeautifulSoup(login_resp.text, 'html.parser')
            csrf_meta = dashboard_soup.find('meta', {'name': 'csrf-token'})
            if csrf_meta:
                dash['csrf_token'] = csrf_meta.get('content')
            else:
                print(f"[{dash['name']}] ⚠️ CSRF token لم يتم العثور على")

            dash['is_logged_in'] = True

```

```

        dash['cookies'] = session.cookies.get_dict()
        return True
    else:
        print(f"[{dash['name']}] ❌ فشل تسجيل الدخول")
        return False

except Exception as e:
    print(f"[{dash['name']}] ❌ خطأ في تسجيل الدخول: {e}")
    return False

# --- دالة جلب الرسائل من iVasms ---
def fetch_ivasms_messages():
    """SMS جلب رسائل iVasms"""
    start_time = time.time() # 🕒 قياس الوقت
    dash = IVASMS_DASHBOARD

    # التأكد من تسجيل الدخول
    if not dash.get('is_logged_in', False):
        if not login_to_ivasms():
            return []

    try:
        session = dash['session']
        base_url = dash['base_url']
        sms_api_url = dash['sms_api_endpoint']
        csrf_token = dash.get('csrf_token')

        if not csrf_token:
            print(f"[{dash['name']}] ⚠️ CSRF token غير متوفر")
            return []

        # إعداد الهيدرز
        headers = {
            'Referer': f"{base_url}/portal/sms/received",
            'X-Requested-With': 'XMLHttpRequest'
        }

        # جلب ملخص الرسائل لآخر 24 ساعة
        today = datetime.utcnow()
        start_date = (today - timedelta(days=1)).strftime('%m/%d/%Y')
        end_date = today.strftime('%m/%d/%Y')

        summary_payload = {
            'from': start_date,
            'to': end_date,
            '_token': csrf_token
        }

```



```

summary_resp = session.post(sms_api_url, headers=headers, data=summary_pa
summary_resp.raise_for_status()

# للوصول إلى مجموعات الدول HTML تحليل ال
summary_soup = BeautifulSoup(summary_resp.text, 'html.parser')
country_groups = summary_soup.find_all('div', {'class': 'pointer'})

if not country_groups:
    print(f"[{dash['name']}] لا توجد مجموعات دول متاحة")
    return []

# استخراج معرفات المجموعات (الدول)
group_ids = []
for group in country_groups:
    onclick = group.get('onclick', '')
    match = re.search(r"getDetials\('(.*?)'\)", onclick)
    if match:
        group_ids.append(match.group(1))

all_messages = []
numbers_url = urljoin(base_url, "portal/sms/received/getsms/number")
sms_details_url = urljoin(base_url, "portal/sms/received/getsms/number/sm

for group_id in group_ids:
    # جلب الأرقام ضمن المجموعة
    numbers_payload = {
        'start': start_date,
        'end': end_date,
        'range': group_id,
        '_token': csrf_token
    }

    numbers_resp = session.post(numbers_url, headers=headers, data=number
    numbers_soup = BeautifulSoup(numbers_resp.text, 'html.parser')

    # استخراج الأرقام
    number_divs = numbers_soup.select("div[onclick*='getDetialsNumber']")
    phone_numbers = [div.text.strip() for div in number_divs]

    for phone in phone_numbers:
        # جلب تفاصيل الرسائل لكل رقم
        sms_payload = {
            'start': start_date,
            'end': end_date,
            'Number': phone,
            'Range': group_id,

```

```

        '_token': csrf_token
    }

    sms_resp = session.post(sms_details_url, headers=headers, data=sms_data)
    sms_soup = BeautifulSoup(sms_resp.text, 'html.parser')

    # استخراج الرسائل
    sms_cards = sms_soup.find_all('div', class_='card-body')

    for card in sms_cards:
        sms_text_p = card.find('p', class_='mb-0')
        if sms_text_p:
            sms_text = sms_text_p.get_text(separator='\n').strip()

            # إنشاء معرف فريد للرسالة
            message_id = f"{phone}-{sms_text[:50]}"

            # استخراج اسم الدولة من group_id
            country_name = group_id.strip()

            all_messages.append({
                'id': message_id,
                'number': phone,
                'text': sms_text,
                'country': country_name,
                'timestamp': datetime.utcnow().isoformat()
            })

    print(f"[{dash['name']}] 

```

```

service = detect_service(sms)

# إذا موجود user_id الحصول على
user_id = get_user_by_number(number)
log_otp(number, otp_code, sms, user_id)

if user_id:
    try:
        markup = types.InlineKeyboardMarkup()
        markup.row(
            types.InlineKeyboardButton("Owner 🇸🇦", url="https://t.me/nolab420")
            types.InlineKeyboardButton("Channel", url="https://t.me/+OQ0CTFKdF")
        )
        bot.send_message(
            user_id,
            f"""\n🌍 <b>Country:</b> {safe_l
            reply_markup=markup,
            parse_mode="HTML"
        )
    except Exception as e:
        print(f"❗ لمستخدم OTP فشل إرسال {user_id}: {e}")

# إرسال نفس الرسالة للجروب
text = format_message(date_str, number, sms)
send_to_telegram_group(text, otp_code)

def delete_message_after_delay(chat_id, message_id, delay=150):
    """تأخير delay ثانية تحذف الرسالة بعد مرور"""
    time.sleep(delay)
    try:
        url = f"https://api.telegram.org/bot{BOT_TOKEN}/deleteMessage"
        payload = {"chat_id": chat_id, "message_id": message_id}
        requests.post(url, data=payload)
    except Exception as e:
        print(f"❌ فشل حذف الرسالة: {e}")

def send_to_telegram_group(text, otp_code):
    success_count = 0
    try:
        # بناء لوحة الأزرار لتتطابق الصورة تماماً
        keyboard = {
            "inline_keyboard": [
                # الزر الأول: نسخ الكود (بشكل عريض)
                [{"text": f"⚡ {otp_code}", "copy_text": {"text": str(otp_code)}}]

                # الصف الثاني: زر القناة وزر لوحة البوت
                [

```

```

        {"text": "💬 CHANNEL PocketMoneyHub ⚡", "url": "https://t.me/PocketMoneyHub"},
        {"text": "PocketMoneyNumber BOT ⚡", "url": "https://t.me/PocketMoneyNumberBOT"}
    ],

    # الصف الثالث: زر المطور
    [
        {"text": "PMH ⚡", "url": "https://t.me/nolab420"}
    ]
]

except Exception as e:
    print(f"❌ خطأ في إعداد الأزرار: {e}")
    return False

url = f"https://api.telegram.org/bot{BOT_TOKEN}/sendMessage"

for chat_id in CHAT_IDS:
    try:
        payload = {
            "chat_id": chat_id,
            "text": text,
            "parse_mode": "HTML",
            "reply_markup": json.dumps(keyboard)
        }
        resp = requests.post(url, data=payload, timeout=10)

        if resp.status_code == 200:
            print(f"[+] تم إرسال الرسالة بنجاح إلى {chat_id}")
            success_count += 1

            # حذف الرسالة بعد 150 ثانية
            msg_id = resp.json()["result"]["message_id"]
            threading.Thread(
                target=delete_message_after_delay,
                args=(chat_id, msg_id, 150),
                daemon=True
            ).start()
        else:
            print(f"[!] فشل إرسال {chat_id}: {resp.status_code}")
    except Exception as e:
        print(f"[!] خطأ في الإرسال لـ {chat_id}: {e}")

return success_count > 0

@bot.callback_query_handler(func=lambda call: call.data.startswith("copy_"))
def handle_copy_button(call):
    otp_code = call.data.split("_", 1)[1]

```

```

bot.answer_callback_query(call.id, f"✅ تم نسخ الكود {otp_code}", show_alert

# PocketMoneyHub | غير معروف callback معالج عام لأي
@bot.callback_query_handler(func=lambda call: True)
def handle_unknown_callback(call):
    """غير معروف callback معالج احتياطي لأي"""
    try:
        print(f"⚠️ Callback معروف: {call.data} من المستخدم {call.from_user.id}")
        bot.answer_callback_query(
            call.id,
            "⚠️ /start، هذا الزر غير متاح حالياً. جرب",
            show_alert=True
        )
    except Exception as e:
        print(f"❌ العام: {e} callback خطأ في معالج الـ")

# PocketMoneyHub | الحلقة الرئيسية
def main_loop():
    global REFRESH_INTERVAL
    REFRESH_INTERVAL = 2 # ⚡ ثواني للفحص السريع 2

    # قائمة باللوحة الوحيدة
    DASHBOARDS = [IVASMS_DASHBOARD]

    # ملف لتخزين معرفات الرسائل المرسله
    SENT_MESSAGES_FILE = "ivasms_sent_messages.json"
    sent_messages = set()

    # تحميل الرسائل المرسله سابقاً إن وجدت
    try:
        if os.path.exists(SENT_MESSAGES_FILE):
            with open(SENT_MESSAGES_FILE, 'r') as f:
                sent_messages = set(json.load(f))
    except Exception as e:
        print(f"⚠️ خطأ في تحميل الرسائل المرسله {e}")

    print("=" * 60)
    print(f"🚀 بدء مراقبة لوحة iVasms (كل 3 ثوان)")
    print("=" * 60)

    consecutive_errors = {dash["name"]: 0 for dash in DASHBOARDS}

    # تسجيل الدخول الأولي
    for dash in DASHBOARDS:
        if not dash.get('is_logged_in', False):
            login_to_ivasms()

```

```

while True:
    for dash in DASHBOARDS:
        try:
            print(f"[{dash['name']}] 🕒 جلب الرسائل...")

            # جلب الرسائل
            messages = fetch_ivasms_messages()

            if messages:
                new_messages = 0
                # معالجة الرسائل من الأحدث إلى الأقدم
                for msg in messages:
                    msg_id = msg['id']

                    if msg_id not in sent_messages:
                        # استخراج البيانات
                        number = clean_number(msg['number'])
                        sms_text = msg['text']
                        date_str = msg['timestamp']

                        # إرسال الرسالة
                        send_otp_to_user_and_group(date_str, number, sms_text)

                        # إضافة إلى قائمة المرسلات
                        sent_messages.add(msg_id)
                        new_messages += 1

                if new_messages > 0:
                    print(f"[{dash['name']}] ✅ تم إرسال {new_messages} جديدة")

                    # حفظ قائمة الرسائل المرسلات
                    try:
                        with open(SENT_MESSAGES_FILE, 'w') as f:
                            json.dump(list(sent_messages)[-1000:], f) # فقط
                    except Exception as e:
                        print(f"⚠️ خطأ في حفظ الرسائل المرسلات {e}")

                    consecutive_errors[dash["name"]] = 0
                else:
                    print(f"[{dash['name']}] [=] لا توجد رسائل جديدة")

            # تنظيف الذاكرة
            if len(sent_messages) > 2000:
                sent_messages = set(list(sent_messages)[-1000:])

        except Exception as e:
            consecutive_errors[dash["name"]] += 1

```

```

        print(f"[{dash['name']}] ❌ خطأ ({consecutive_errors[dash['name']]  

        if consecutive_errors[dash["name"]] >= 5:  

            print(f"[{dash['name']}] 🚫 إعادة تسجيل الدخول بعد 5 أخطاء")  

            dash['is_logged_in'] = False  

            login_to_ivasms()  

            consecutive_errors[dash["name"]] = 0  

  

        time.sleep(REFRESH_INTERVAL)  

  

# PocketMoneyHub | تشغيل البوت التفاعلي في خيط منفصل  

def run_bot():  

    print("[*] Starting bot...")  

    while True:  

        try:  

            print("[*] Bot polling started...")  

            bot.polling(none_stop=True, timeout=60)  

        except Exception as e:  

            print(f"❌ Bot polling error: {e}")  

            traceback.print_exc()  

            print("[*] Restarting bot in 5 seconds...")  

            time.sleep(5)  

  

if __name__ == "__main__":  

    threading.Thread(target=run_bot, daemon=True).start()  

    main_loop()

```